

Handout: 725103 Data Structure and Algorithm

สัปดาห์ที่ 2: Pseudocode • การวิเคราะห์ big-O

ภาคปลาย ปีการศึกษา 2568 | อาจารย์ผู้สอน: ปพนธ์ร์ แยมโชติ

เป้าหมายการเรียนรู้ประจำสัปดาห์ (Learning Outcomes)

เมื่อจบคาบนี้ นักศึกษาสามารถ...

- อ่าน pseudocode เข้าใจ และเขียน trace การทำงานในแต่ละขั้นของ pseudocode ได้
- เข้าใจความหมายของ big-O ในการวิเคราะห์ความซับซ้อนของการทำงานของ program ผ่าน pseudocode ได้ (แยกได้ระหว่างความซับซ้อนและความเร็วในการรัน)

1 Pseudocode

Pseudocode คือการเขียนการทำงานของโปรแกรม โดยใช้ภาษามนุษย์เพื่อเป็นไกด์ไลน์ไว้บอกตรรกะที่ออกแบบไว้โดยไม่ขึ้นอยู่กับภาษาโปรแกรมที่จะเลือกใช้ (ดังนั้น จึงไม่มีกฎเกณฑ์ตายตัวในการเขียน แต่ในวันนี้จะเน้นการอ่านให้เข้าใจก่อน แล้วหลังจากคุณเขียน Python ในวิชา 725102 คล่องแล้ว จะนำไปลองเขียนดูก็ได้) โครงสร้างที่จะใช้ในวันนี้จะเขียนดังนี้

Algorithm 1 ชื่อฟังก์ชัน หรือชื่ออัลกอริทึม

Require: เขียนตัวแปรและความหมายของตัวแปร

1: (เขียนขั้นตอน)

- เลขบรรทัด เขียนมาเพื่อให้ระบุจุดที่อ้างถึงตอนสอนได้สะดวก ตอนเขียนเอง นักศึกษาจะไม่เขียนเลขบรรทัดก็ได้
- ในหลายๆ ภาษาต้องมีการประกาศประเภทของตัวแปรก่อน แต่เราไม่จำเป็นต้องระบุประเภทของตัวแปรก็ได้ เนื่องจากเราใช้ Python ที่ไม่เข้มงวดเรื่องประเภทตัวแปร (แต่ถ้าใช้ภาษาที่เข้มงวด ก็ควรเขียนกำกับไว้เพื่อเป็นการเตือนตัวเอง)

แนวคิดสำคัญของ pseudocode

- **State** ของโปรแกรม = ค่าปัจจุบันของตัวแปรทั้งหมด (เช่น x, y, sum, i)
- **Trace** = โล่บรรทัดแล้วอัปเดตค่า State ที่ละขั้น (เหมือนดูการหมุนที่ละเฟรม)
- **Branching** (if/else) = เลือกทำ "ทางใดทางหนึ่ง" ตามเงื่อนไข
- **Loop** (for/while) = ทำซ้ำจนกว่าจะครบรอบหรือเงื่อนไขหยุด

1.1 Warm-up: โครงสร้างควบคุมการทำงาน (Control Flow)

- | | |
|--------------------------|---|
| 1: $x =$ ค่าอะไรสักอย่าง | ▷ กำหนดค่าให้ตัวแปร |
| 2: $x = a + b$ | ▷ ด้านขวาสามารถเป็นก้อนของการคำนวณได้ ซึ่งจะคำนวณก่อนแล้วค่อยกำหนดค่าให้ตัวแปร |
| 3: $x = x + 1$ | ▷ สามารถคำนวณค่าจากตัวแปรหนึ่งๆ แล้วกำหนดกลับไปให้ตัวแปรเดิมก็ได้ (ไม่ใช่สมการ) |

Example 1. จงเขียนค่าของตัวแปรที่ทั้งหมดที่มีในแต่ละขั้นตอน

Algorithm 2 ตัวอย่างการกำหนดค่า

```
1:  $x = 5$   
2:  $y = x + 1$   
3:  $y = 2 * y$ 
```

คำตอบ/การทำงานทีละขั้น:

- หลังบรรทัด 1: $x =$ _____ $y =$ _____
- หลังบรรทัด 2: $x =$ _____ $y =$ _____
- หลังบรรทัด 3: $x =$ _____ $y =$ _____

ในวันนี้ นักศึกษาไม่ต้อง ``เขียนโปรแกรมจริง" แต่ต้อง **อ่าน** และ **trace** ให้ได้ก่อน เพราะในวิชานี้เราจะอธิบายอัลกอริทึมผ่าน pseudocode เป็นหลัก

Example 2. พิจารณาว่ากระบวนการนี้ทำอะไร

Algorithm 3 ตัวอย่างการกำหนดค่า

Require: x และ y คือจำนวนเต็ม

```
1:  $temp = y$   
2:  $y = x$   
3:  $x = temp$ 
```

1.1.1 if/else: เลือกทางเดิน

Example 3. จง trace ค่า ans ทีละขั้น เมื่อ (1) ให้ $x = -3$ (2) ให้ $x = 5$

Algorithm 4 AbsoluteValue

Require: x คือจำนวนเต็ม

```
1: if  $x \geq 0$  then  
2:    $ans = x$   
3: else  
4:    $ans = -x$   
5: end if  
6: return  $ans$ 
```

1.1.2 for: ทำซ้ำจำนวนรอบที่รู้ล่วงหน้า

รูปแบบที่เราจะใช้บ่อยคือ for $i = 1$ to n (นับจาก 1 ถึง n รวมปลายทั้งคู่)

Example 4. ให้ $n = 5$ จงเติมตาราง trace ให้ครบ

Algorithm 5 SumToN

Require: n คือจำนวนเต็มบวก

```
1:  $sum = 0$ 
2: for  $i = 1$  to  $n$  do                                ▷ บางครั้งก็เขียนว่า  $i = 1..n$ 
3:    $sum = sum + i$ 
4: end for
5: return  $sum$ 
```

i	sum หลังจบบรรทัด $sum = sum + i$
1	
2	
3	
4	
5	

1.1.3 while: ทำซ้ำจนกว่าจะหยุด

while จะทำซ้ำ ``ตราบใดที่" เงื่อนไขยังเป็นจริง (ดังนั้น ต้องระวัง สัมผัสให้เงื่อนไขเปลี่ยน แล้ววนไม่จบ)

Example 5. ให้ $n = 3$ โปรแกรมจะพิมพ์อะไรออกมาบ้าง (เขียนเป็นลำดับบรรทัด)

Algorithm 6 CountDown

Require: n คือจำนวนเต็มไม่ติดลบ

```
1: while  $n > 0$  do
2:    $print(n)$ 
3:    $n = n - 1$ 
4: end while
5:  $print("Done")$ 
```

1.2 การ trace อย่างเป็นระบบ: Checklist กันพลาด

เวลา trace ให้ทำตามลำดับนี้เสมอ

1. เขียนค่าเริ่มต้นของตัวแปรทุกตัวที่เกี่ยวข้อง (State เริ่มต้น)
2. ไต่บรรทัดตามเลขบรรทัด: ``บรรทัดนี้ทำอะไร" แล้วอัปเดต State
3. เจอ if: ตัดสินใจด้วยเงื่อนไข (จริง/เท็จ) แล้วไปทางที่ถูก
4. เจอ loop: ระบุ ``ตัวแปรนับรอบ" และเขียนเป็นตาราง trace ช่วย

คำเตือนที่พบบ่อย

- สับสน ระหว่าง ``เท่ากับ" ในคณิตศาสตร์ กับ ``กำหนดค่า" ในโปรแกรม: $x = x + 1$ อ่านว่า ``เอา x เดิมบวก 1 แล้วเก็บกลับใน x "
- สิม ว่า for $i = 1$ to n ทำทั้งหมด n รอบ (รวมทั้ง $i = 1$ และ $i = n$)
- สิม เปลี่ยนตัวแปรใน while ทำให้วนไม่จบ (infinite loop)

2 คณิตศาสตร์พื้นฐานสำหรับการวิเคราะห์ (Math Warm-up)

เราจะใช้คณิตศาสตร์แค่ ``ระดับจำเป็น" เพื่อคุยเรื่องการเติบโตของเวลา (ไม่เน้นพิสูจน์ยาก ๆ)

2.1 สรุปสัญลักษณ์และผลรวมที่ใช้บ่อย

- ผลรวม $1 + 2 + \dots + n = \frac{n(n+1)}{2}$
- ถ้าลูปรทำงาน n รอบ และในแต่ละรอบทำงานคงที่ $\Rightarrow$ รวมเป็น ``ประมาณ cn " (โตตาม n)
- ถ้าลูบซ้อนกัน (nested loop) เช่น ทำ n รอบ และข้างในทำ n รอบ $\Rightarrow$ รวมเป็น ``ประมาณ cn^2 "

2.2 แนวคิดของ $\log n$ แบบไม่ใช่แคลคูลัส

- $\log_2 n$ คือ ``จำนวนครั้งที่หาร 2 จนเหลือ 1" (ปัดขึ้น/ลงได้ตามบริบท)
- ตัวอย่าง: $n = 8$ หาร 2 สามครั้ง: $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ ดังนั้น $\log_2 8 = 3$

3 Big-O และการนับจำนวน operation

3.1 ทำไมไม่พูดแค่ว่า ``เร็ว" หรือ ``ช้า"?

ความเร็วจริงขึ้นกับเครื่อง, ภาษา, ตัวแปรแวดล้อม แต่ **Big-O** สนใจ ``แนวโน้มเมื่อข้อมูลโตขึ้น" ดังนั้น Big-O เหมาะกับการเปรียบเทียบวิธีแบบเป็นระบบ (โดยไม่ต้องรันจริง)

Definition 6 (Big-O (แนวคิด)). ให้ $T(n)$ แทนจำนวน operation ที่อัลกอริทึมทำเมื่ออินพุตมีขนาด n เราพูดว่า

$$T(n) \in O(f(n))$$

เมื่อมีค่าคงที่ c และ n_0 ที่ทำให้สำหรับทุก $n \geq n_0$ มี $T(n) \leq cf(n)$

อ่านแบบนักศึกษา (ไม่ต้องท่องนิยาม)

- $O(1)$ = ทำงาน ``คงที่" ไม่ขึ้นกับ n
- $O(n)$ = ทำงานเพิ่มตามขนาดข้อมูลแบบเส้นตรง
- $O(n^2)$ = โตเร็วขึ้นมาก (เช่น loop ซ้อน loop)
- $O(\log n)$ = โตช้า (เช่น ลดปัญหาค้างหนึ่งทุกครั้ง)

3.2 เราจะนับ operation อะไรบ้าง? (Cost Model)

ในวิชานี้ เราจะนับแบบง่าย (เพื่อให้ฝึกได้เร็ว) โดยถือว่าแต่ละอย่างด้านล่าง = 1 operation

- การกำหนดค่า (assignment): $x = \dots$
- การคำนวณเลขพื้นฐาน 1 ครั้ง: $+, -, *, /$
- การเปรียบเทียบ (comparison): $<, <=, ==, >=, >$

หมายเหตุ: การนับแบบนี้ไม่ใช่มาตรฐานเดียวในโลกจริง แต่เพียงพอสำหรับการเข้าใจ Big-O ในระดับพื้นฐาน

3.3 ตัวอย่าง 1: การบวกเลข 1 ถึง n

Algorithm 7 SumToN (นับ operation)

Require: n คือจำนวนเต็มบวก

```
1:  $sum = 0$ 
2: for  $i = 1$  to  $n$  do
3:    $sum = sum + i$ 
4: end for
5: return  $sum$ 
```

การนับแบบคร่าว ๆ ดังนั้น $T(n) \approx$ _____

Exercise 7. ถ้าไม่ใช้วิธีการนับละเอียด อะไรในอัลกอริทึมนี้ที่บอกไว้ว่าเป็น $O(n)$

3.4 ตัวอย่าง 2: loop ซ้อน loop

Algorithm 8 CountPairs

Require: n คือจำนวนเต็มบวก

```
1:  $count = 0$ 
2: for  $i = 1$  to  $n$  do
3:   for  $j = 1$  to  $n$  do
4:      $count = count + 1$ 
5:   end for
6: end for
7: return  $count$ 
```

แนวความคิดการนับ ดังนั้น $T(n) \approx$ _____

Exercise 8. ถ้าเปลี่ยนลูปด้านในให้วิ่งแค่ถึง $j = i$ (คือ **for** $j = 1$ to i) จะเหลือประมาณกี่ครั้ง? และเป็น $O(\cdot)$ อะไร?

Algorithm 9 CountPairs

Require: n คือจำนวนเต็มบวก

```
1:  $count = 0$ 
2: for  $i = 1$  to  $n$  do
3:   for  $j = 1$  to  $i$  do
4:      $count = count + 1$ 
5:   end for
6: end for
7: return  $count$ 
```

3.5 ตัวอย่าง 3: ลดปัญหาครึ่งหนึ่งทุกครั้ง

Algorithm 10 HalfUntilOne

Require: n คือจำนวนเต็มบวก

```
1:  $count = 0$ 
2: while  $n > 1$  do
3:    $n = \lfloor n/2 \rfloor$ 
4:    $count = count + 1$ 
5: end while
6: return  $count$ 
```

คำถาม: ลูปทำกี่รอบ?

ดังนั้น $T(n) \approx$ _____

Exercise 9. ให้ $n = 20$ จง trace ค่า n ในแต่ละรอบ จนลูปหยุด และเขียนจำนวนรอบทั้งหมด

Algorithm 11 Legendre Formula: ไว้ใช้หาว่า $n!$ ลงท้ายด้วย 0 กี่ตัว

Require: n คือจำนวนเต็มบวก

```
1:  $count = 0$ 
2: while  $n > 1$  do
3:    $n = \lfloor n/5 \rfloor$ 
4:    $count = count + n$ 
5: end while
6: return  $count$ 
```

ดังนั้น $T(n) \approx$ _____

Exercise 10. ให้ $n = 1000$ จง trace ค่า n ในแต่ละรอบ จนลูปหยุด และเขียนจำนวนรอบทั้งหมด

3.6 ตัวอย่าง 4: เปรียบเทียบกับ best/worst case

Algorithm 12 FindFirstZero

Require: $A[1..n]$ คืออาร์เรย์ของจำนวนเต็ม

```
1: for  $i = 1$  to  $n$  do
2:   if  $A[i] == 0$  then
3:     return  $i$ 
4:   end if
5: end for
6: return  $-1$ 
```

▷ ไม่พบ

- **Best case:** เจอศูนย์ตั้งแต่ตัวแรก ($i = 1$) $\Rightarrow$ _____
- **Worst case:** ไม่มีศูนย์เลย หรืออยู่ตัวสุดท้าย $\Rightarrow$ _____

Exercise 11. ถ้าในอาร์เรย์มีศูนย์อยู่เสมอ ``เฉลี่ย" จะเป็น $O(\cdot)$ อะไร?

4 แบบฝึกในคาบ: จากการนับ operation $\rightarrow$ สรุปเป็น Big-O

โจทย์ A (Warm-up)

Algorithm 13 A

Require: n คือจำนวนเต็มบวก

```
1:  $x = 0$ 
2: for  $i = 1$  to  $n$  do
3:    $x = x + 2$ 
4: end for
5: return  $x$ 
```

Exercise 12. (1) x จะได้ค่าเท่าไรเมื่อจบ? (2) อัลกอริทึมนี้เป็น $O(\cdot)$ อะไร?

โจทย์ B (นับรอบไม่เท่ากัน)

Algorithm 14 B

Require: n คือจำนวนเต็มบวก

```
1:  $count = 0$ 
2: for  $i = 1$  to  $n$  do
3:   for  $j = 1$  to  $2n$  do
4:      $count = count + 1$ 
5:   end for
6: end for
7: return  $count$ 
```

Exercise 13. จงนับจำนวนครั้งที่บรรทัดในสุดทำงาน (ในรูปของ n) และสรุปเป็น $O(\cdot)$

โจทย์ C (รูปเพิ่มทีละ 2)

Algorithm 15 C

Require: n คือจำนวนเต็มบวก

```
1:  $i = 1$ 
2: while  $i \leq n$  do
3:    $i = i + 2$ 
4: end while
```

Exercise 14. รูปนี้ทำงานกี่รอบประมาณ? และเป็น $O(\cdot)$ อะไร?

โจทย์ D (สามเหลี่ยม)

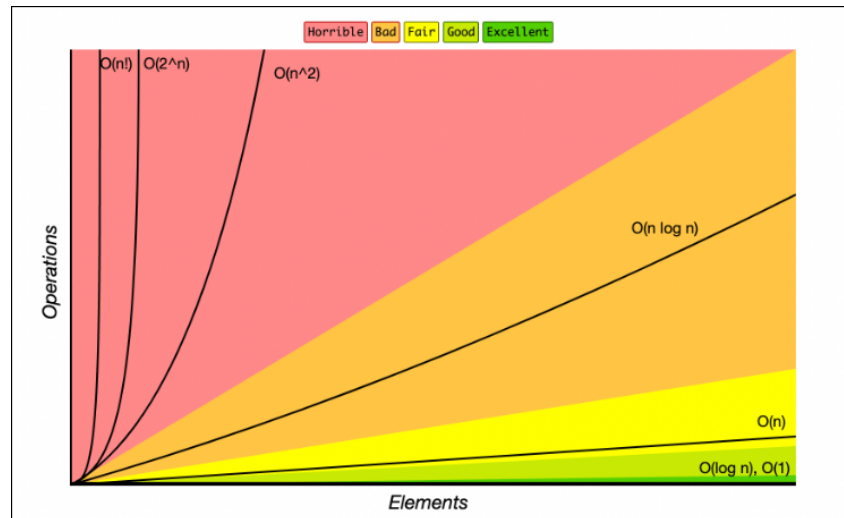
Algorithm 16 D

Require: n คือจำนวนเต็มบวก

```
1:  $s = 0$ 
2: for  $i = 1$  to  $n$  do
3:   for  $j = 1$  to  $i$  do
4:      $s = s + 1$ 
5:   end for
6: end for
7: return  $s$ 
```

Exercise 15. (1) บรรทัดในสุดทำทั้งหมดกี่ครั้ง (เขียนเป็นผลรวม) (2) สรุปเป็น $O(\cdot)$

5 สรุปกราฟการเติบโตที่ควรรู้จัก



รูปแบบ	มาจากอะไรบ่อย	ตัวอย่างแนวคิด
$O(1)$	ทำงานคงที่	เข้าถึง $A[i]$ 1 ครั้ง
$O(\log n)$	ลดครึ่งทุกครั้ง	binary search, ทหาร 2 জনหมด
$O(n)$	ทำทีละตัวครบทุกตัว	เดินดูทุกคนในคิว
$O(n \log n)$	แบ่งครึ่ง + ทำซ้ำหลายชั้น	divide-and-conquer หลายแบบ
$O(n^2)$	loop ซ้อน loop	เปรียบเทียบทุกคู่
$O(2^n), O(n!)$	มีจำนวนงานย่อยแบบไม่คอยลดลง (โซนนันตราย)	เล่น Tower of Hanoi

จำไว้ 1 ประโยค

Big-O บอก "การโตเมื่อ n โต" ไม่ได้บอก "เวลาจริงเป็นวินาที" และ complexity ต่ำกว่า ไม่ได้หมายความว่าเร็วหรือช้าเสมอ แต่บอกว่าจะช้าลงช้ากว่า (คล้ายกับการมีอัตราการเสื่อมถอยช้ากว่า)

6 การบ้านหลังเรียน (สั้น 10--15 นาที)

- ทำโจทย์ A--D ให้ครบ (ถ้ายังไม่เสร็จในคาบ)
- "ทำไม $O(n)$ บางครั้งรันจริงอาจเร็วกว่า $O(\log n)$?"